Publishing IMU Data Topics

1. Experiment Objective

In this experiment, you will learn how to publish the ROS2 standard `sensor_msgs/msg/Imu` message through micro-ROS, including acceleration, angular velocity, and the quaternion orientation obtained through AHRS attitude fusion. Through this example, you will master reading the QMI8658B six-axis IMU sensor, the Fusion AHRS attitude fusion algorithm, ROS time synchronization, and a multi-task cooperation pattern that uses critical sections to protect shared sensor data. This example publishes the topic `/imu` at 20 Hz.

2. Experiment Principle

1. Keyword Explanation

Concept	Description
IMU	Inertial Measurement Unit, a sensor module integrating accelerometers and gyroscopes. The MicroROS-V2 control board carries the on-board QMI8658B six-axis IMU, which can measure three-axis acceleration and three-axis angular velocity
sensor_msgs/Imu	The ROS2 standard IMU message type, defined in `sensor_msgs/msg/Imu`, containing the quaternion orientation, angular velocity, linear acceleration and their covariance matrices
Quaternion	A mathematical tool that represents 3D rotation with four parameters (x, y, z, w), avoiding the gimbal lock problem of Euler angles, and is the standard form of orientation representation in ROS2. A quaternion satisfies `x² + y² + z² + w² = 1`
Angular velocity	The rate at which an object rotates about each axis, in rad/s. In IMU data, angular_velocity represents the rotation angular velocity around the X, Y and Z axes and can be measured directly by the gyroscope
AHRS attitude fusion	Using the complementary characteristics of accelerometer and gyroscope data, a complete 3D attitude is computed through an algorithm (the Mahony filter in this example), overcoming the drift of pure gyroscope integration and the poor dynamic response of the accelerometer
Critical section	A task synchronization mechanism provided by FreeRTOS, protecting shared data access through `taskENTER_CRITICAL` / `taskEXIT_CRITICAL`, preventing data races between the high-priority fusion task and the publishing task

2. Technical Architecture

This example adopts a **dual-task + critical section shared data** design, separating high-speed sensor fusion (100 Hz) from ROS2 publishing (20 Hz) to ensure the real-time performance of data acquisition and the stability of publishing.

Configuration Item	Value	Description
Node name	`imu_publisher`	ROS2 node name
Topic name	`imu`	Topic for publishing IMU data
Message type	`sensor_msgs/msg/Imu`	ROS2 standard IMU message
Publish frequency	20 Hz	Timer period 50 ms
QoS mode	Best Effort	High-frequency large messages reduce transmission burden
frame_id	`imu_frame`	IMU coordinate frame identifier

Data flow: **QMI8658B sensor → imu_fusion_task (100 Hz AHRS fusion, writes shared data in a critical section) → micro_ros_task timer callback (20 Hz reads the shared data and publishes it)**.

3. Core Topic Quick Reference

Firmware Uplink (Sensor Data)

Topic	Message Type	QoS	Description
`/imu`	`sensor_msgs/msg/Imu`	Best Effort	IMU data (quaternion orientation + angular velocity + linear acceleration), published at 20 Hz

Usage Example:

```
ros2 topic echo /imu
```

4. Middleware Configuration

micro-ROS uses a **static memory model** (`RCUTILS_AVOID_DYNAMIC_ALLOCATION=ON`), all entity memory is pre-allocated at compile time, and runtime dynamic creation is not supported.

> Configuration file: `~/esp/extra_components/micro_ros_espidf_component/colcon.meta`

`RMW_UXRCE_TRANSPORT`	`udp`	UDP transport (Wi-Fi communication)
`RMW_UXRCE_MAX_PUBLISHERS`	`6`	Maximum number of publishers
`RMW_UXRCE_MAX_SUBSCRIPTIONS`	`6`	Maximum number of subscribers
`RMW_UXRCE_MAX_NODES`	`1`	Maximum number of nodes
`RMW_UXRCE_MAX_SERVICES`	`0`	Services not supported
`RMW_UXRCE_MAX_CLIENTS`	`0`	Clients not supported

`RMW_UXRCE_MAX_HISTORY`	`1`	History depth = 1, no message queuing
`UCLIENT_UDP_TRANSPORT_MTU`	`4096`	Maximum UDP packet size 4 KB
`UCLIENT_MIN_HEARTBEAT_TIME_INTERVAL`	`1`	Agent heartbeat interval 1 ms

Key Constraints:

- `MAX_HISTORY=1` means there is no message buffer queue, so uplink topics (firmware → host) uniformly use **best_effort** QoS
- `MAX_SERVICES=0, MAX_CLIENTS=0` means only topic communication is supported, not Service/Client
- After modifying `colcon.meta`, the micro-ROS library must be recompiled for the changes to take effect: run `idf.py clean-microros` in the project directory to clean and rebuild the micro-ROS library, then `idf.py build` to compile the firmware

3. Experiment Steps

1. Hardware Preparation

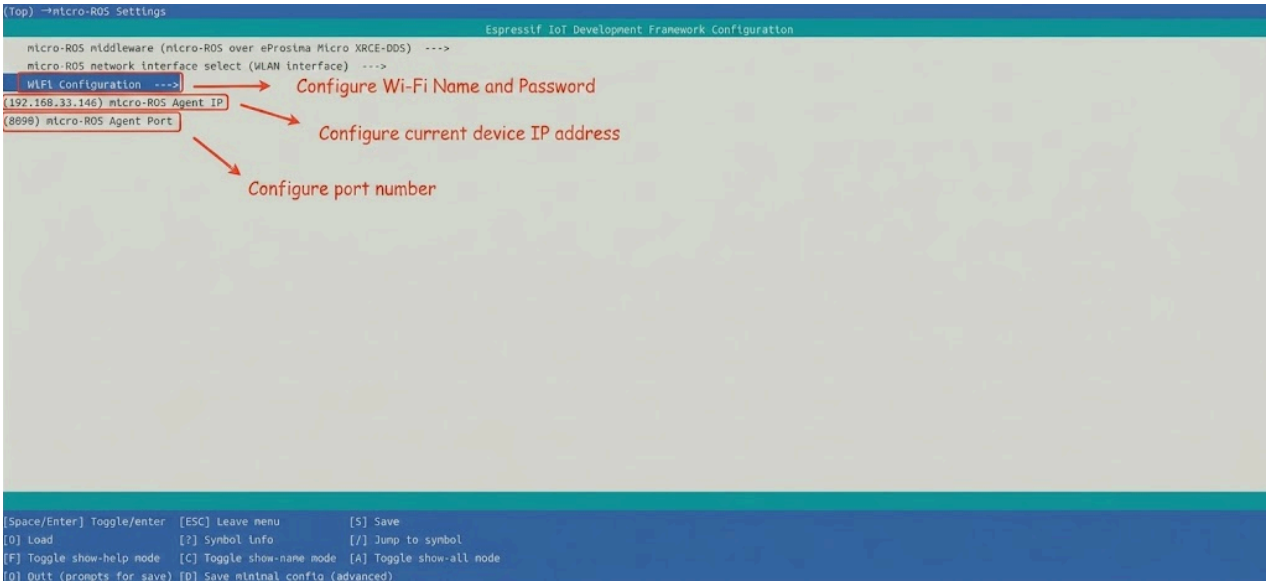
Hardware	Requirement
MicroROS-V2 control board	✅ Required
USB Type-C cable	✅ Required
Wi-Fi network (2.4 GHz)	✅ Required

> The MicroROS-V2 control board carries the on-board QMI8658B six-axis IMU sensor, so no external hardware is required.

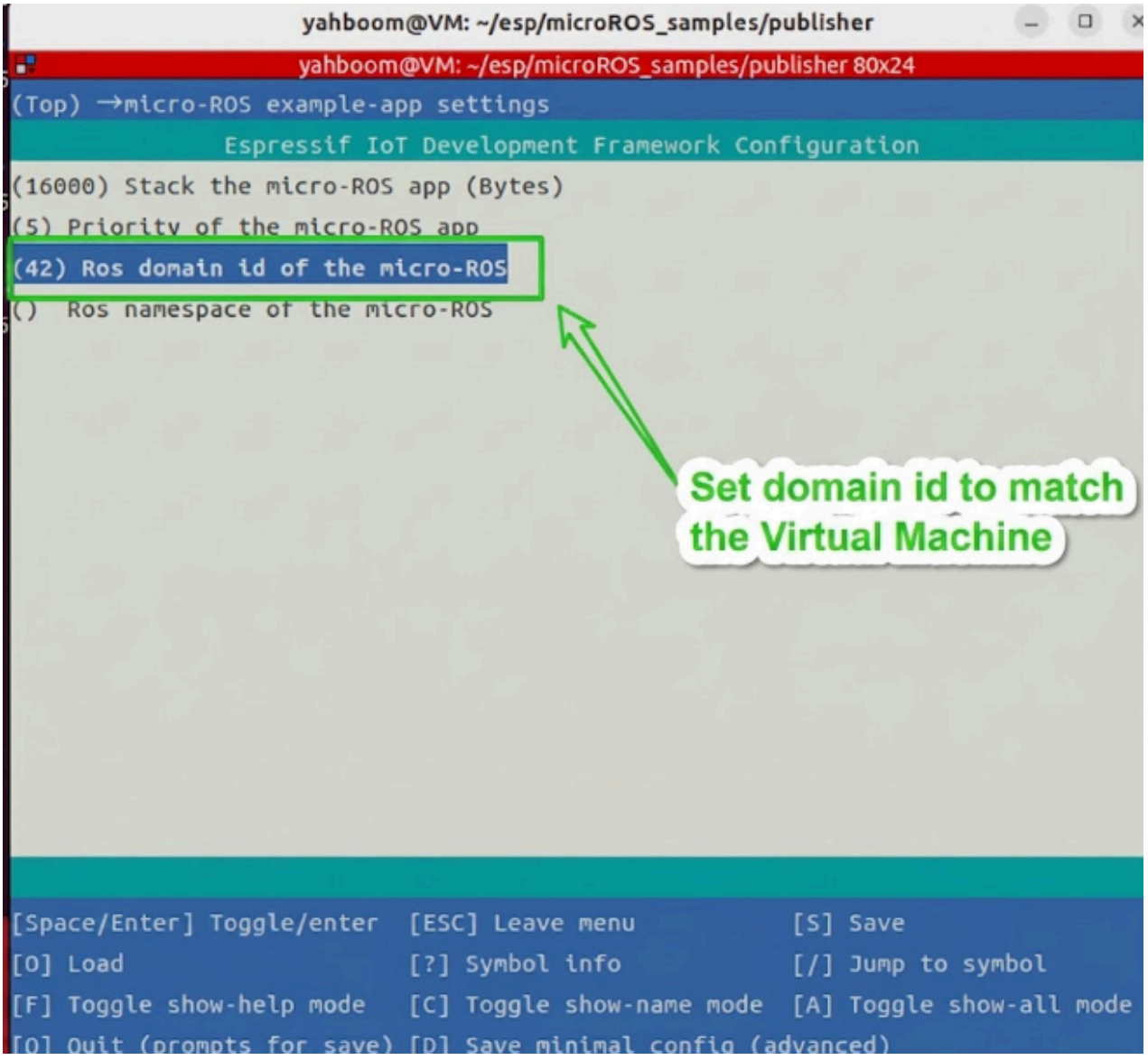
2. Prerequisites

1. The ESP32-IDF development environment has been set up
2. The MicroROS-V2 control board is connected to the computer via USB
3. Configure the Wi-Fi SSID, password, Agent IP and port via `idf.py menuconfig` (the `micro_ros` menu item)

```
cd ~/esp/microROS_samples/imu_publisher
source ~/esp/esp-idf/export.sh
idf.py menuconfig
```



4. Select example-app setting to set the domain ID to be consistent with the virtual machine



After configuration, press "s" to save and press "q" to exit the configuration interface.

3. Start micro_ros_agent (Terminal 1)

Start the micro-ROS Agent on the ROS2 host:

```
cd ~/uros_ws
source install/local_setup.bash
ros2 run micro_ros_agent micro_ros_agent udp4 --port 8090 -v4
```

4. Build and Flash (Terminal 2)

```
cd ~/esp/microROS_samples/imu_publisher
source ~/esp/esp-idf/export.sh
idf.py build flash monitor
```

5. How to Stop

1. Press `Ctrl+]` in Terminal 2 to exit the serial monitor (stop firmware execution)
2. Press `Ctrl+C` in Terminal 1 to stop micro_ros_agent

4. Experiment Result

After the build and flash succeed, view the IMU data on the ROS2 host:

```
ros2 topic echo /imu
```

Expected output (updated at 20 Hz, containing quaternion orientation, angular velocity, linear acceleration):

```
header:
  stamp:
    sec: xxx
    nanosec: xxx
  frame_id: imu_frame
orientation:
  x: 0.001
  y: -0.002
  z: 0.710
  w: 0.704
angular_velocity:
  x: 0.001
  y: -0.003
  z: 0.052
linear_acceleration:
  x: -0.152
  y: 0.087
  z: 9.801
```

The control board serial port will output:

```
I (xxx) MAIN: Agent reachable: 192.168.x.x:8888
I (xxx) MAIN: time synced, offset=xxx ms
I (xxx) MAIN: IMU fusion task started
I (xxx) MAIN: micro-ROS IMU publisher init done
```

Data verification methods:

- When the control board is placed horizontally, `linear_acceleration.z` should be approximately equal to 9.8 m/s² (gravitational acceleration)
- Rotating the control board around the Z axis, `orientation.z` and `orientation.w` should change accordingly (representing the yaw angle)
- When stationary, each axis of `angular_velocity` should be close to 0

5. Program Analysis

> Source code paths:
>
> - Firmware source code: `~/esp/microROS_samples/imu_publisher/main/main.c`
> - IMU fusion: `~/esp/microROS_samples/imu_publisher/components/imu_attitude_fusion/imu_attitude_fusion.h`, `imu_attitude_fusion.c`

1. Main Program Logic

`main/main.c` adopts a **dual-task + critical section shared data** design:

1. In `app_main()`:
   - Calls `imu_ros_init()` to pre-initialize the IMU message structure
   - Initializes the Wi-Fi network
   - Creates `imu_fusion_task` (high priority, 6144-byte stack space), waiting for the ROS-ready signal
   - Creates `micro_ros_task`
2. In `imu_fusion_task` (running at 100 Hz):
   - Waits for the ROS initialization to complete (blocking wait via `ulTaskNotifyTake`)
   - Initializes the Fusion AHRS algorithm and gyroscope bias calibration
   - Reads the QMI8658B sensor data every 10 ms
   - Calls `imu_attitude_fusion_update()` to perform the AHRS fusion
   - Writes the fusion result to the global shared struct `g_imu_shared` (protected by a critical section)
3. In `micro_ros_task`:
   - Creates the node `imu_publisher`
   - Publishes the topic `imu` in Best Effort mode, with message type `sensor_msgs/msg/Imu`
   - Creates a 50 ms timer (20 Hz), obtains the latest IMU data from the shared data and publishes it
   - Performs ROS time synchronization every 10 seconds

2. Key Components

IMU hardware parameters:

> Parameter definition file: `~/esp/microROS_samples/imu_publisher/main/main.c`

Parameter	Value	Description
Sensor model	QMI8658B	Six-axis inertial measurement unit (3-axis accelerometer + 3-axis gyroscope)
Communication interface	I2C	Reads sensor data through the I2C bus
Fusion sampling rate	100 Hz	`IMU_FUSION_SAMPLE_RATE_HZ = 100.0`
Gyroscope range	1024 dps	`IMU_FUSION_GYRO_RANGE_DPS = 1024.0`
Publish frequency	20 Hz	`IMU_PUBLISH_PERIOD_MS = 50 ms`
Fusion update period	10 ms	`IMU_UPDATE_PERIOD_MS = 10 ms`

`components/imu_attitude_fusion/imu_attitude_fusion.h` — IMU fusion interface:

Function	Description
`imu_attitude_fusion_init(FusionAhrs* fusion, FusionBias* bias)`	Initializes the AHRS fusion algorithm
`fusion_param_setting(...)`	Sets the sampling rate and gyroscope range parameters
`calibrate_gyro_startup_bias()`	Calibrates the gyroscope zero offset at startup
`update_imu_data(FusionVector* acc, FusionVector* gyro)`	Reads the raw QMI8658B data
`imu_attitude_fusion_update(...)`	Performs one AHRS fusion update
`imu_attitude_fusion_get_quaternion(FusionAhrs* fusion)`	Gets the fused quaternion orientation
`imu_attitude_fusion_get_euler(quaternion)`	Converts a quaternion to Euler angles (degrees)

`components/Fusion/` — Fusion AHRS algorithm library:

Based on the open-source xioTechnologies/Fusion sensor fusion library, it provides AHRS attitude estimation based on the accelerometer and gyroscope. It uses the Mahony filter algorithm, which is computationally efficient and suitable for microcontroller platforms.

IMU mounting orientation compensation:

Because the IMU chip is mounted on the PCB rotated 180° around the Z axis relative to the front of the car, the code negates the X and Y axes of the accelerometer and gyroscope as compensation:

> Source file: `~/esp/microROS_samples/imu_publisher/main/main.c`

```
static inline void imu_apply_mount_rotation_180deg_z(FusionVector *acc_raw,
                                                     FusionVector *gyro_raw)
{
    if (acc_raw != NULL) {
        acc_raw->axis.x = -acc_raw->axis.x;
        acc_raw->axis.y = -acc_raw->axis.y;
    }
    if (gyro_raw != NULL) {
        gyro_raw->axis.x = -gyro_raw->axis.x;
        gyro_raw->axis.y = -gyro_raw->axis.y;
    }
}
```

Unit conversion:

- Gyroscope: raw value (dps) → `× DEG_TO_RAD` → rad/s
- Accelerometer: raw value (g) → `× GRAVITY_STANDARD` → m/s²

Critical section shared data pattern:

> Source file: `~/esp/microROS_samples/imu_publisher/main/main.c`

```
typedef struct {
    double angular_velocity_x, angular_velocity_y, angular_velocity_z;
    double linear_acceleration_x, linear_acceleration_y, linear_acceleration_z;
    double orientation_x, orientation_y, orientation_z, orientation_w;
    double euler_roll_deg, euler_pitch_deg, euler_yaw_deg;
    bool valid;
} imu_shared_data_t;

// Written by the fusion task
taskENTER_CRITICAL(&g_imu_lock);
g_imu_shared = local;
taskEXIT_CRITICAL(&g_imu_lock);

// Read by the publish callback
taskENTER_CRITICAL(&g_imu_lock);
snapshot = g_imu_shared;
taskEXIT_CRITICAL(&g_imu_lock);
```

Timer callback `timer_imu_callback`:

> Source file: `~/esp/microROS_samples/imu_publisher/main/main.c`

```
static void timer_imu_callback(rcl_timer_t *timer, int64_t last_call_time)
{
    RCLC_UNUSED(last_call_time);
```

```
    if (timer == NULL) {
        return;
    }

    imu_shared_data_t snapshot;

    // Read the shared data in a critical section
    taskENTER_CRITICAL(&g_imu_lock);
    snapshot = g_imu_shared;
    taskEXIT_CRITICAL(&g_imu_lock);

    if (!snapshot.valid) {
        return;
    }

    // Fill the ROS timestamp
    const struct timespec time_stamp = get_timespec();
    msg_imu.header.stamp.sec = (int32_t)time_stamp.tv_sec;
    msg_imu.header.stamp.nanosec = (uint32_t)time_stamp.tv_nsec;

    // Fill the IMU data
    msg_imu.angular_velocity.x = snapshot.angular_velocity_x;
    msg_imu.angular_velocity.y = snapshot.angular_velocity_y;
    msg_imu.angular_velocity.z = snapshot.angular_velocity_z;

    msg_imu.linear_acceleration.x = snapshot.linear_acceleration_x;
    msg_imu.linear_acceleration.y = snapshot.linear_acceleration_y;
    msg_imu.linear_acceleration.z = snapshot.linear_acceleration_z;

    msg_imu.orientation.x = snapshot.orientation_x;
    msg_imu.orientation.y = snapshot.orientation_y;
    msg_imu.orientation.z = snapshot.orientation_z;
    msg_imu.orientation.w = snapshot.orientation_w;

    // Publish the IMU message
    const rcl_ret_t rc = rcl_publish(&publisher_imu, &msg_imu, NULL);
    if (rc != RCL_RET_OK) {
        imu_pub_fail_cnt++;
        if ((imu_pub_fail_cnt % 20U) == 1U) {
            print_rcl_error("rcl_publish(&publisher_imu, &msg_imu, NULL)",
                            __LINE__, rc);
        }
    }
}
```

6. Common Problems

Problem	Cause	Solution
Publish failure, empty IMU data	QMI8658B initialization failed	Check the I2C bus connection and confirm the sensor is not occupied by another task

Problem	Cause	Solution
Attitude data drift	The gyroscope zero offset was not calibrated correctly	`calibrate_gyro_startup_bias()` is automatically executed at startup; ensure the control board is kept stationary during startup
Euler angle value jumps	Gimbal lock problem	The quaternion data is not affected; using the quaternion is recommended
Unstable publish frequency	Priority contention between the fusion task and the publish task	The fusion task priority is slightly higher than the micro-ROS task, ensuring data updates are not blocked
Abnormal accelerometer Z-axis value	Incorrect mounting orientation compensation	The current code compensates for a 180° rotation around the Z axis; if the actual mounting orientation differs, adjust `imu_apply_mount_rotation_180deg_z`